



Coláiste na Tríonóide, Baile Átha Cliath
Trinity College Dublin

Ollscoil Átha Cliath | The University of Dublin

Faculty of Science, Technology, Engineering and Mathematics

School of Computer Science & Statistics

Senior Freshman
Computer Science and Language
Computer Science and Business

Semester 2 2024

2062 - Natural Language Processing

Friday 3rd May

GOLDSMITH HALL

14:00 – 16:00

Dr. Martin Emms

Instructions to Candidates:

There are **4** questions. Answer any **2**

Each question is worth 50 marks

There is a **Supplementary Information** sheet giving some information required for particular questions.

1. Question concerns implementation of bottom-up stack-based parser

- (a) To implement a shift-reduce parser some stack-like component is required. The C++ vector might serve as a basis for this. Accordingly consider the class (the *Context Free Grammar* classes are assumed):

```
class CatStk {
public:
    void push(Category c);
    Category pop();
    Category top();
    bool is_empty();
    bool top_match(vector<Category> cats);
    int size();

private:
    vector<Category> pda;
};
```

- i. Implement `Category CatStk::top()` and `Category CatStk::pop()` [5 marks]
 - ii. Implement `CatStk::top_match(..)` so that when passed the *dtrs* part of a rule, it returns true if the *top-most* elements of the stack match, with the *right-most daughter* corresponding to the *top* of the stack. [10 marks]
- (b) Suppose the following classes for implementing a backtracking shift-reduce parser; Fork records parser states to which it can return to seek alternative ways to proceed.

<pre>class BU { public: Grammar g; vector<string> input; int progress; int rule_index; CatStk cat_stk; };</pre>	<pre>class BU_BT : BU { public: vector<Fork> fork_stk; bool accept(vector<string> s); int move(); };</pre>	<pre>class Fork { public: Fork(); CatStk cat_stk; int rule_index; int progress; };</pre>
---	--	--

- i. Figure 1 gives partial code for the `BU_BT::accept(..)` method to give it the behaviour of a *bottom-up, backtracking parser*. **Complete this code.**

You may assume the `move()` method returns 1 if a reduction can be made using a Rule whose index is $\geq$ `rule_index`, and 2 if a shift can be made, and else returns 0. In case 1, `rule_index` is set to the index of the relevant rule.

[30 marks]

(question continues next page)

```

bool BU_BT::accept(vector<string> key) {

    bool backtracking = true;
    initialize cat_stk, fork_stk, rule_index, progress, input

    do {
        while((move_type = move()) != 0) {
            if(move_type indicates reduction) {
                update fork_stk, cat_stk, progress;
                rule_index=0;
            }
            else if(move_type indicates shift) {
                update cat_stk, progress;
                rule_index = 0;
            }
        }

        if (accepting configuration) {
            accept = true;
            break;
        }
        else if (fork_stk is not empty) {
            set cat_stk, progress, rule_index from fork_stk;
            update fork_stk;
        }
        else {
            backtracking = false;
            accept = false;
        }
    } while(backtracking);

    return accept;
}

```

Figure 1: Outline of bottom-up backtracking algorithm

- ii. Supposing the grammar below, the backtracking *top-down* parser will have a termination problem on the input *walks quickly*. Give a trace of its execution in this case, including commentary on the backtracking involved. [5 marks]

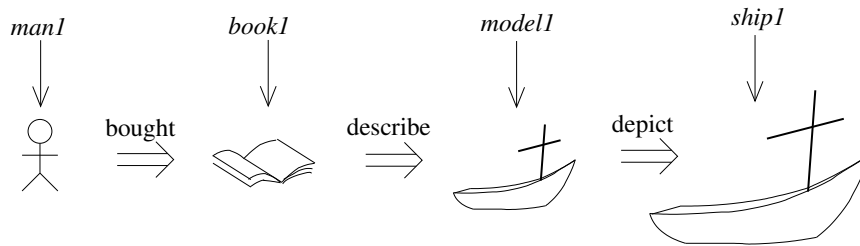
```

vp --> [runs]
vp --> vp ,adv
adv --> [quickly]

```

2. Question concerns relative clauses and Slash-grammars

(a)



- i. Referring to the above picture, *man1* can be identified by mentioning *book1* in a *subject* relative clause: '*the man that bought **book1***'; *book1* can in its turn be identified via its relation to *model1* resulting in:

*the man that bought the book that described **model1***

The language $\mathcal{NP}SRC$ defined below is arrived at by thinking about repeatedly replacing the final identifier with a description involving a subject relative clause:

$$\begin{aligned}\mathcal{NP}SRC &= \{(\text{the } N \text{ that } TV)^n PN \mid n \geq 0\} \text{ where} \\ N &\in \{man, book, model, ship, \} \quad TV \in \{bought, described, depicted\} \\ PN &\in \{man1, book1, model1, ship1\}\end{aligned}$$

Give a CFG which also defines the set $\mathcal{NP}SRC$. [4 marks]

- ii. Give also an FSA which defines $\mathcal{NP}SRC$. [4 marks]

- iii. Working instead with *object* relative clauses, *model1* can be identified by mentioning *book1* in an object relative clause: '*the model that **book1** described*'; *book1* can be identified in a corresponding way via its relation to *man1* resulting in

*the model that the book that **man1** bought described*

The language $\mathcal{NP}ORC$ defined below is arrived at by thinking about repeatedly replacing the innermost identifier with a description involving an object relative clause:

$$\begin{aligned}\mathcal{NP}ORC &= \{(\text{the } N \text{ that})^n PN (TV)^n \mid n \geq 0\} \text{ where} \\ N, TV, PN &\text{ as for } \mathcal{NP}SRC\end{aligned}$$

Give a CFG which also defines the set $\mathcal{NP}ORC$. [5 marks]

- iv. Give the analysis tree according to your grammar of

the ship that the model that book1 described depicted [4 marks]

- v. The language $\{a^n b^n \mid n \geq 1\}$ is very analogous to $\mathcal{NP}ORC$. Outline the proof using the Pumping Lemma (see **Supplementary Info**) which shows that $\{a^n b^n \mid n \geq 1\}$ *could not* be defined by an FSA [8 marks]

- (b) The correct generalisation about relative clauses is that what follows the relative pronoun ought to be

any sentence from which some noun-phrase has been removed, thereby leaving a 'gap' **(Gap Gen)**

The 'slash grammars' approach to this derives rules involving 'slash categories' from rules which do not, in such a way that s/np categorises strings that fit **Gap Gen**.

Starting from the union of the grammars $\mathcal{G}_{start}$ and $\mathcal{G}_{rel}$ below, derive in this way a 'slash grammar' allowing for nps with relative clauses.

Make sure to cover the following, drawing their trees according to your grammar:

the ship that he told her to buy

the woman that he told to buy the ship

the ship that the model that book1 started to describe depicted [25 marks]

$\mathcal{G}_{start}$:

s --> np, vp	np --> [ship1]
	np --> [model1]
vp --> tv, np	np --> [book1]
vp_b --> tv_b, np	np --> [man1]
	np --> [he]
vp --> v2, tovp	np --> [she]
vp_b --> v2_b, tovp	np --> [him]
	np --> [her]
vp --> v3, np, tovp	np --> det, n
vp_b --> v3_b, np, tovp	det --> [the]
tovp --> [to], vp_b	n --> [ship]
	n --> [model]
tv --> [bought]	n --> [book]
tv_b --> [buy]	n --> [man]
tv --> [described]	n --> [woman]
tv_b --> [describe]	
tv --> [depicted]	
tv_b --> [depict]	
v2 --> [started]	
v2_b --> [start]	
v3 --> [told]	
v3_b --> [tell]	

$\mathcal{G}_{rel}$:

np --> det, n, relpro, s/np
relpro --> [which]
relpro --> [that]

3. Question concerns agreement in FSAs and CFGs

- (a) In English there are some *agreement* requirements concerning the determiner word at the beginning of a noun-phrase and the noun at the end of a noun-phrase, as illustrated by

many grumpy indolent sloths

many grumpy indolent sloth *

each grumpy indolent sloths *

each grumpy indolent sloth

Let $\mathcal{L}$ be defined as the following fragment of English illustrating this

every expression in $\mathcal{L}$

begins with a member of {many, each}

is followed by $n \geq 0$ members of {grumpy, indolent}

and ends with

a member of {sloths, koalas} if the initial word was many

a member of {sloth, koala} if the initial word was each

Give an FSA which defines $\mathcal{L}$

[10 marks]

- (b) CFGs enhanced with *feature structures* allow such phenomena to be described concisely.

- (i) The idea that one feature structure can *specialise* into another is a key one, and in terms of it the notion of *unification* of two feature structures can be defined. Give such a definition.

[4 marks]

- (ii)(A) Give an example pair of unifiable non-identical feature structures FS_1 and FS_2 , at least one of which has two attributes, showing their unification.

(B) Similarly give an example of a non-unifiable pair.

- (iii) Give a grammar $\mathcal{G}_1$ in the form a CFG enhanced with feature structures which defines the above-described language $\mathcal{L}$. Don't worry about the particular notation in which the rules are expressed, so long as it clearly pairs category names with feature structures.

[16 marks]

- (iv) In using a featurized grammar, if its rules do not mention a particular feature at all in association with a particular word, does that exclude that word's appearance in position's where a particular value for that feature is required?

[4 marks]

(question continues next page)

- (v) Unlike *many* and *each*, the word *the* does not place requirements on the number of the final noun:

the grumpy indolent sloths

the grumpy indolent sloth

Extend your grammar to $\mathcal{G}_2$ to deal with this also, and say in a few words how specialisation and unification permit the above to be parsed when $\mathcal{G}_2$ is used.

[8 marks]

- (vi) In the course both untyped and typed feature structures were discussed. Briefly discuss a possible advantage to the use of typed feature structures. [8 marks]

4. Question concerns computing a semantic value

Note: refer to the **Semantics** section in the *Supplementary Information*

(a) Suppose p is a Rel object to represent arity 1 relations. What will the contents of p be on executing the following:

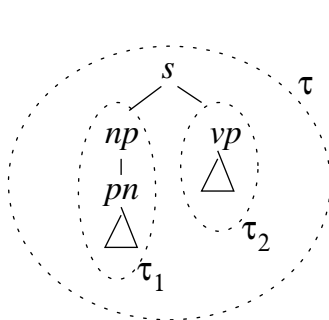
i. $p = \text{reduce}(\text{likes}, 1, \text{"maths"})$; [2 marks]

ii. $p = \text{reduce}(\text{likes}, 0, \text{"ron"})$; [2 marks]

(b) Provide code implementing $\text{Rel reduce}(\text{Rel } r, \text{int } which, \text{Thing } v)$, using the helper function $\text{Tuple erase}(\text{Tuple } t, \text{int } i)$ [5 marks]

(c)(i) Provide code implementing $\text{bool true_of}(\text{Rel } p, \text{Thing } o)$ which when applied to 1-place relation p , and some Thing o , returns true just when the 1-tuple consisting just of o is one of those in p . [3 marks]

(ii) Complete the parts enclosed in `/* .. */` in the partial semantic evaluation function for the structure $s(\text{np}(\text{pn}(\dots)), \text{vp}(\dots))$: [7 marks]



```
bool compute_s_value(Tree *tau) {
    bool v;
    if(tau, tau_1, tau_2 are as shown to the left) {
        /* code using compute_np_value(..)
           compute_vp_value(..)
           true_of(..) */
        return v;
    }
    else { poss other cases }
}
```

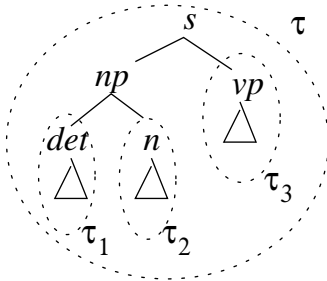
(d)(i) Provide code implementing the following functions ‘

A. $\text{bool some}(\text{Rel } p, \text{Rel } q)$: returns true when at least one tuple x which is in p 's tuples is in q 's tuples also [5 marks]

B. $\text{bool every}(\text{Rel } p, \text{Rel } q)$: returns true when every tuple x which is in p 's tuples is in q 's tuples also [5 marks]

(question continues next page)

- (ii) Complete the parts enclosed in `/* .. */` in the functions below to handle structures of the form `s(np(det( $D$ ), n(...)), vp(...))` for $D \in \{a, every\}$: [12 marks]



```
bool compute_s_value(Tree * $\tau$ ) {
    :
    else if( $\tau, \tau_1, \tau_2, \tau_3$  are as in the pic. to left){
        string  $D$  = get_word( $\tau_1$ );
        Rel v1, v2;
        /* code using compute_n_value( $\tau_2$ )
           compute_vp_value( $\tau_3$ )
           gen_quant( $D, v1, v2$ ) */
    }
}
```

```
bool gen_quant(string  $D$ , Rel p, Rel q) {
    /* code using some(..), every(..)
       for  $D == "a"$  and  $D == "every"$  */
}
```

(assume `get_word( $\tau$ )` when applied to a `det`-tree returns the leaf of that tree)

- (e) Consider

- (α) every student respires
- (β) Ron is a student
- (γ) Ron respires

If we suppose for structures `vp(is,a,n(..))` that the `vp`'s value is that of its `n` part, the concerted effect of the semantic evaluation functions is such that

No matter what the settings of the relations, it will never be the case that (α) and (β) get value true whilst (γ) gets value false.

Show that this is the case.

[9 marks]